SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Reviewe2Revenue

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Table of Content **2**
1. Introduction: **3**
1.1. Purpose: 3
1.2. Background: 3
1.3. Target Stakeholder 4
2. System Architecture & Design **5**
2.1. High-Level Architecture 5
2.1.1. Overview: 5
2.1.2 LLM as Service Layer 5
2.1.3 Dependency Diagram 5
3. Functional Requirements & Scope **8**
3.1. Minimum Viable Product: 8
3.2. Non-Functional Requirements (NFRs) 9
3.3. Out of Scope / Future Enhancements 9
4. Monitor, Evaluation, Assumptions & Dependencies **10**
4.1. Technical Evaluation 10
4.1.1. Grayscale Rollout & A/B Testing: 10
4.1.2. Emergency Rollback (ER) & Golden Release 10
4.1.3. Priority Matrix 10
4.2. Monitoring 12
4.3. Assumptions 12
4.4. External Dependencies 12
5. Project Management & Team Contributions **12**
5.1. Project Timeline 12
5.2. Team Members' Roles 13
5.3. Recommendations 14

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces "Review2Revenue," a role-based AI restaurant assistant designed to solve problems in restaurant discovery and business analysis.

1.1. Purpose:

The system analysis document highlights the technical scope and design decisions behind the Review2Revenue development. The key elements of the system covered are as followed:

1. **Architecture**:
    a. It is a client-server system using a React frontend, a FastAPI backend, and multi-provider LLM routing. The application logic is separated by user role (Diner vs. Vendor) within the shared backend.
2. **Data Flows:**
    a. Data, including user profiles, restaurant metadata, and reviews, is stored in a PostgreSQL/Supabase database. User requests move through the backend API, where context is gathered from the database, routed to an external LLM (Groq, Gemini, or Z.AI), and the final structured response is returned to the client.
3. **Model Process:**
    a. The model shows the end-to-end workflow, including user login via Firebase, profile resolution in Supabase, permanent role selection (Diner/Vendor), and role-specific AI request processing (Diner recommendations or Vendor business analysis).
4. **Role of Reference:**
    a. This document is a reference for developers and testers to understand the overall high-level architecture, implementation, and defined scope of the Review2Revenue project.

1.2. Background:

The Review2Revenue project addresses the need for a targeted, AI-driven restaurant intelligence platform. It provides personalized discovery for diners and evidence-grounded operational insights for restaurant vendors based on review and sentiment analysis.

1. Previous Version:
    a. This is the initial version of the Review2Revenue platform, focusing on building a minimum viable product (MVP) with distinct, permanent role-based paths for Diners and Vendors.
2. Changes in Major Architectural Components
    a. The architecture incorporates a modern stack: React for the frontend, FastAPI for the backend, PostgreSQL/Supabase for data persistence, and a multi-provider LLM router for AI inference.
3. New capabilities that are only introduced in this

© UMHackathon 2026

a. New capabilities include permanent role selection during registration, vendor restaurant locking (tying a vendor to one store_id), a multi-provider LLM fallback system for reliability, and dedicated CSV upload features for vendor reviews and menus.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Diner	Browse restaurants, search, and submit natural-language dining requests to the AI recommendation assistant.	Simple discovery experience, ranked recommendations with explanations, and location-aware search.
Vendor	Manage and analyze their assigned restaurant using review data and sentiment insights.	Actionable insights for quality questions (diagnostics, strengths, improvements) and the ability to upload review/menu data.
Development Team	Build and maintain the platform (Frontend, Backend, Database, LLM routing).	Clear API usage contracts, a modular codebase, well-documented architecture, and type safety.
QA Team	Validating system behavior for both Diner and Vendor roles, ensuring correct AI responses and data isolation.	Defined scope, clear request/response formats, coverage of role-specific guardrails, and LLM fallback behavior.

© UMHackathon 2026

2. System Architecture & Design

2.1. High-Level Architecture

2.1.1. Overview:

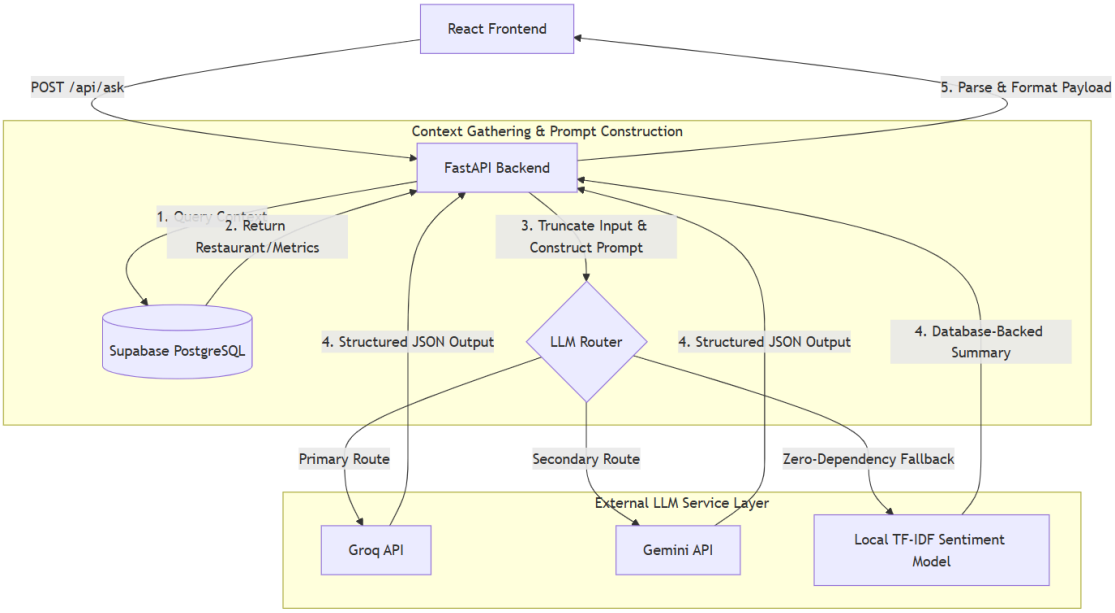
Type	Details
System	Web Application (React frontend, FastAPI backend)
Architecture	Client-Server with Multi-Provider LLM Integration

Review2Revenue is primarily structured as a client-server-based system. There are two primary clients (Diner Web App and Vendor Web App). All clients communicate with the FastAPI Backend API layer. The backend is responsible for role-aware handlers, data access (PostgreSQL/Supabase), and utilizing a unified router for three independent LLM providers: Groq, Gemini, and Z.AI.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram

2.2. Technological Stack

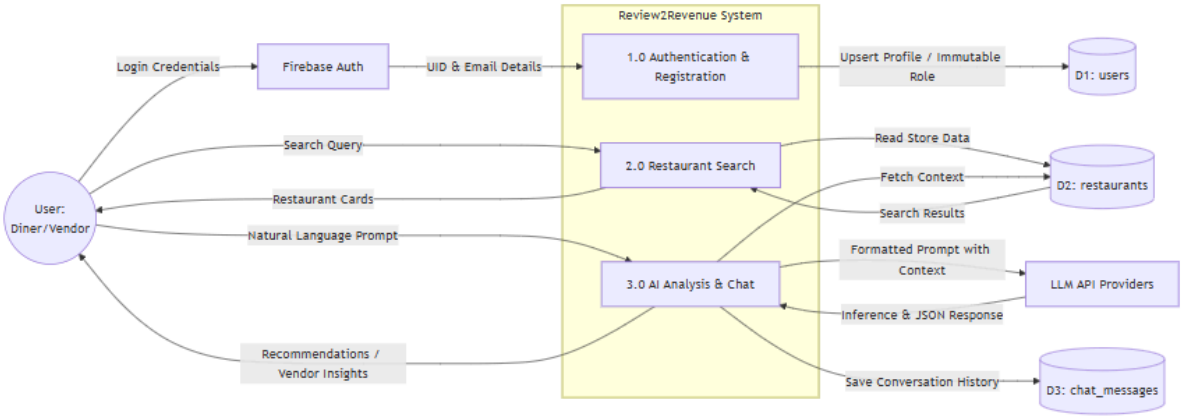
© UMHackathon 2026

Frontend: React + Vite.2
Backend: FastAPI + SQLAlchemy (RESTful API for the microservices).2
Database: PostgreSQL/Supabase (Relational Database for users, restaurants, reviews, and chat history).24
Cloud/Deployment: Firebase Authentication (Google provider) and Supabase Postgres.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

```
-- 1. Create independent tables first

CREATE TABLE restaurants (
    store_id TEXT PRIMARY KEY,
    name TEXT,
    food_type TEXT,
    google_place_id TEXT,
    google_formatted_address TEXT,
    google_price_tier TEXT,
    google_lat FLOAT8,
    google_lng FLOAT8,
    google_website TEXT,
    google_phone TEXT
);

-- 2. Create tables that depend only on 'restaurants'

CREATE TABLE users (
    id TEXT PRIMARY KEY,
```

© UMHackathon 2026

```
    user_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    role TEXT NOT NULL,
    store_id TEXT REFERENCES restaurants(store_id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE restaurant_operating_hours (
    id INT8 GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    store_id TEXT REFERENCES restaurants(store_id) ON DELETE CASCADE,
    day_of_week INT4 CHECK (day_of_week BETWEEN 1 AND 7), -- 1=Monday, 7=Sunday
    open_time TIME,
    close_time TIME,
    is_closed BOOLEAN DEFAULT FALSE
);

CREATE TABLE menu_items (
    menu_id TEXT PRIMARY KEY,
    store_id TEXT REFERENCES restaurants(store_id) ON DELETE CASCADE,
    item_name TEXT,
    category TEXT,
    price_rm FLOAT8,
    source TEXT,
    is_available BOOLEAN DEFAULT TRUE,
    updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE reviews (
    uuid TEXT PRIMARY KEY,
    store_id TEXT REFERENCES restaurants(store_id) ON DELETE CASCADE,
    updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    review_text TEXT,
    overall_rating INT8,
    food_rating INT8,
    rider_rating FLOAT8,
    sentiment TEXT,
    sentiment_model TEXT
);

-- 3. Create conversation & messaging tables (depend on 'users')

CREATE TABLE conversations (
    conversation_id TEXT PRIMARY KEY,
    user_id TEXT REFERENCES users(id) ON DELETE CASCADE,
    title TEXT,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE chat_messages (
```

© UMHackathon 2026

```
    id INT8 GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    conversation_id TEXT REFERENCES conversations(conversation_id) ON DELETE CASCADE,
    role TEXT NOT NULL, -- e.g., 'user', 'assistant', 'system'
    message TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

-- 4. Create the junction table (depends on 'chat_messages' and 'restaurants')

CREATE TABLE message_restaurant_references (
    message_id INT8 REFERENCES chat_messages(id) ON DELETE CASCADE,
    store_id TEXT REFERENCES restaurants(store_id) ON DELETE CASCADE,
    PRIMARY KEY (message_id, store_id)
);

-- 5. Create the Materialized View for performance metrics

CREATE MATERIALIZED VIEW restaurant_metrics AS
SELECT
    store_id,
    AVG(overall_rating) AS avg_rating,
    COUNT(*) AS total_reviews,
    ROUND((SUM(CASE WHEN sentiment = 'positive' THEN 1 ELSE 0 END)::NUMERIC /
NULLIF(COUNT(*), 0)), 2) AS positive_ratio,
    ROUND((SUM(CASE WHEN sentiment = 'negative' THEN 1 ELSE 0 END)::NUMERIC /
NULLIF(COUNT(*), 0)), 2) AS negative_ratio
FROM
    reviews
GROUP BY
    store_id;

-- Optional: Create a unique index to allow concurrent refreshes of the materialized view later
CREATE UNIQUE INDEX idx_restaurant_metrics_store_id ON restaurant_metrics(store_id);
```

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. **[e.g. in Table]**

#	Features	Description

1	Role-Based Access Control	Immutable routing during registration dividing users into Diners (discovery) and Vendors (business insights).
2	Diner AI Recommendations	Diners submit natural language prompts to receive AI-ranked restaurant suggestions with rendering cards.
3	Vendor Business Intelligence	Vendors view dashboard metrics strictly locked to their assigned store_id, analyzing local and external reviews.
4	LLM Fallback System	f external AI providers fail, the backend seamlessly falls back to a locally trained TF-IDF + Logistic Regression sentiment model.

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Scalability	Case-insensitive restaurant searches must respond quickly to prevent UI lag.	Implementation of a 300ms debounce on the frontend and PostgreSQL ILIKE operators on the backend.
Reliability	The application must remain stable even if third-party LLM APIs experience downtime.	A zero-dependency database-summary fallback and local sentiment model catch network errors and return local insights.
Maintainability	Vendor accounts must not exist without a valid physical restaurant mapping.	Database constraints enforce store_id rules, preventing Supabase upserts if the restaurant is not found.

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for “Grand Vision” but not part of this UMHackathon 2026.

© UMHackathon 2026

a. Post-registration role switching between Diner and Vendor.

b. Live payment processing, table reservations, or in-app delivery tracking.

c. Multi-restaurant management capabilities for franchise owners.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout & A/B Testing:

To minimize disruption and ensure system stability, new major features—such as enhanced AI routing logic or new Vendor dashboard widgets—will follow a grayscale rollout strategy. Initial deployments via Netlify (Frontend) and Render (Backend) will be released to 10% of active users. Key performance indicators, including API latency and AI output parsing success rates, will be monitored for 24 hours. If no anomalies are detected, traffic routing will gradually increase to 50%, and finally 100%.

Once a feature reaches full deployment, A/B testing will be conducted to optimize user experience. For example, two variants of the AI recommendation system (Variant A: highly detailed text rationale; Variant B: concise text with prominent UI tags) will be served to two equal user cohorts to determine which strategy yields higher Diner engagement and satisfaction.

4.1.2. Emergency Rollback (ER) & Golden Release

Review2Revenue utilizes an automated CI/CD pipeline that strictly tags and maintains a "Golden Release"—the last verified, highly stable build of both the Vite frontend and FastAPI backend. In the event of an emergency, such as a cascading failure in the multi-LLM router or a critical Foreign Key constraint violation within the Supabase database post-deployment, the Emergency Rollback (ER) protocol is triggered.

Upon triggering, traffic management automatically halts the current deployment and instantly reverts the active production environments on Render and Netlify to the Golden Release. This ensures the system remains highly available to Diners and Vendors while developers isolate and patch the underlying defect in a staging environment.

4.1.3. Priority Matrix

Priority Level	Description & Examples	Trigger Condition	Recommended Action

© UMHackathon 2026

P0 (Critical)	Core system failure prevents basic usage. Examples include Firebase Authentication outages, Supabase database connection loss, or complete failure of the core /api/ask endpoint.	> 2% failure rate or HTTP 500 errors across user sessions within a 5-minute window.	Trigger Emergency Rollback (ER) to the Golden Release immediately. Page the core backend engineering lead.
P1 (High)	Degradation of core features. Examples include total timeout of external LLM APIs (Groq/Gemini/Z.AI) or failure to fetch Google Places data.	100% failure of external API routing requiring fallback activation.	Verify that the system has successfully degraded to the local TF-IDF sentiment fallback model and database summaries. Investigate third-party API quotas and status pages.
P2 (Medium)	Non-critical feature bugs. Examples include chat history failing to render visually on the ChatPage.jsx component or missing UI animations (e.g., floating food emojis).	Identified during routine QA testing or flagged via user bug reports.	Log the defect in the project management board (Jira/Trello). Prioritize and assign for resolution within the next bi-weekly development sprint.
P3 (Low)	Minor UI/UX inconveniences or typos. Examples include slight CSS misalignment on mobile views or minor grammar issues in standard error banners.	Identified during code review or minor user feedback.	Add to the product backlog to be addressed during routine maintenance or UI polish phases.

© UMHackathon 2026

4.2. Monitoring

System health is monitored via the /health FastAPI endpoint. Uvicorn logs track API latency, while the Supabase dashboard is utilized to monitor Foreign Key constraint violations and raw database performance.

4.3. Assumptions

State key operational & environmental conditions that you assumed to be valid during the development of the system and deployment. [e.g.

a. Vendors are attempting to register for restaurants that have already been seeded or synthesized within the PostgreSQL database.

b. Users have stable internet connections to load the Vite frontend and authenticate via Google pop-up windows.

4.4. External Dependencies

Tools	Purpose	Risks
LLM APIs (Groq, Gemini, Z.AI)	It is the core LLM reasoning engine that handles prompt inference, multi-step reasoning, and generating structured output.	High: In case of API rate limitation or total failure, the core product will degrade, falling back to a deterministic, rule-based response.
Firebase Authentication	Handling Google sign-in and user authentication to provide secure access to the platform.	Medium: If Firebase is down, users cannot log in or register, halting all application functionality.
Supabase/ PostgreSQL	Provides persistent storage for all core data, including user profiles, restaurant metadata, reviews, and chat history.	High: Loss of connectivity or data corruption would prevent the system from functioning.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline

Duration	Activities

© UMHackathon 2026

Day 1-2	Project Planning, Architecture Design, Data Model (Supabase Schema and Migrations).
Day 3-5	Authentication (Firebase Google Auth) and Registration Flow Implementation (Role-based routing and Supabase upsert).
Day 6-8	Backend API Development (LLM routing, Ranking, Vendor Analysis logic) and Core Frontend UI (Diner/Vendor views, Chat, Data Upload).
Day 9-10	Testing (Happy Path and Error Case Test Cases), Comprehensive Documentation, and Final Presentation Prep.

5.2. Team Members' Roles

Team Member	Role & Key Responsibilities
Tan Guan Zhe	**Backend Lead & AI Integration** • Architected the FastAPI microservices and the multi-provider LLM router (Groq/Gemini/Z.AI). • Developed the local TF-IDF + Logistic Regression sentiment fallback model. • Managed core data logic, prompt construction, and /api/ask routing.
Eng Yi Fan	**Frontend Lead (React & UI/UX)** • Developed the Vite + React frontend application utilizing glassmorphism design principles. • Integrated Firebase Google Authentication and immutable Diner/Vendor role state management. • Built the interactive chat interface, debounced search features, and dynamic restaurant cards.
Poh Kok Hao	**Database & Cloud Architecture** • Designed the normalized PostgreSQL database schema (3NF) hosted on Supabase. • Implemented strict Foreign Key constraints

© UMHackathon 2026

	(e.g., locking vendors to a store_id) and Row Level Security (RLS) policies. • Managed continuous deployment pipelines via Render (Backend) and Netlify (Frontend).
Tee Wei Hon	**Quality Assurance & Data Engineering** • Authored the Quality Assurance Testing Documentation (QATD) and managed CI/CD automation gates. • Handled data pipelines, including synthetic metadata generation and Google Places API enrichment. • Executed API load testing, edge-case prompt testing, and verified the LLM hallucination/fallback mechanisms.

5.3. Recommendations

Redis Caching Layer: Implement a Redis cache for read-heavy, infrequently changing data. Caching static restaurant metadata, menu items, and popular search results (/api/restaurants/search) will drastically reduce the load on the Supabase database and drop API response times to sub-50ms.

Database Search Optimization: The current system relies on PostgreSQL's case-insensitive ILIKE operator for restaurant searches. As the restaurant's table grows, this will become a bottleneck. We recommend applying a pg_trgm (trigram) index to the name column to maintain ultra-fast, typo-tolerant search performance.

Connection Pooling: As FastAPI scales horizontally to handle more Diner web traffic, it can quickly exhaust active database connections. Implement Supabase's native PgBouncer integration to pool and manage database connections efficiently.

© UMHackathon 2026